

条款 25：考虑写出一个不抛异常的 swap 函数

Consider support for a non-throwing swap.

swap 是个有趣的函数。原本它只是 STL 的一部分，而后成为异常安全性编程（exception-safe programming，见条款 29）的脊柱，以及用来处理自我赋值可能性（见条款 11）的一个常见机制。由于 swap 如此有用，适当的实现很重要。然而在非凡的重要性之外它也带来了非凡的复杂度。本条款探讨这些复杂度及因应之道。

所谓 *swap*（置换）两对象值，意思是将两对象的值彼此赋予对方。缺省情况下 *swap* 动作可由标准程序库提供的 swap 算法完成。其典型实现完全如你所预期：

```
namespace std {
    template<typename T>           //std::swap 的典型实现;
    void swap( T& a, T& b)         //置换 a 和 b 的值.
    {
        T temp(a);
        a = b;
        b = temp;
    }
}
```

只要类型 *T* 支持 *copying*（通过 *copy* 构造函数和 *copy assignment* 操作符完成），缺省的 swap 实现代码就会帮你置换类型为 *T* 的对象，你不需要为此另外再做任何工作。

这缺省的 swap 实现版本十分平淡，无法刺激你的肾上腺。它涉及三个对象的复制：*a* 复制到 *temp*，*b* 复制到 *a*，以及 *temp* 复制到 *b*。但是对某些类型而言，这些复制动作无一必要；对它们而言 swap 缺省行为等于是把高速铁路铺设在慢速小巷弄内。

其中最主要的就是“以指针指向一个对象，内含真正数据”那种类型。这种设计的常见表现形式是所谓“pimpl 手法”（pimpl 是 “pointer to implementation” 的缩写，见条款 31）。如果以这种手法设计 *Widget class*，看起来会像这样：

```
class WidgetImpl {                //针对 Widget 数据而设计的 class;
public:                           //细节不重要。
    ...
private:
    int a, b, c;                  //可能有许多数据,
    std::vector<double> v;        //意味复制时间很长。
    ...
};
```

```

class Widget {                                //这个 class 使用 pimpl 手法
public:
    Widget(const Widget& rhs);
    Widget& operator=(const Widget& rhs)      //复制Widget时, 令它复制其
    {                                         // WidgetImpl 对象。
        ...                                 //关于 operator=的一般性实现
        *pImpl = *(rhs.pImpl);             // 细节, 见条款 10,11 和 12。
        ...
    }
    ...
private:
    WidgetImpl* pImpl;                       //指针, 所指对象内含
};                                           // Widget 数据。

```

一旦要置换两个 Widget 对象值, 我们唯一需要做的就是置换其 pImpl 指针, 但缺省的 swap 算法不知道这一点。它不只复制三个 Widgets, 还复制三个 WidgetImpl 对象。非常缺乏效率! 一点也不令人兴奋。

我们希望能够告诉 std::swap: 当 Widgets 被置换时真正该做的是置换其内部的 pImpl 指针。确切实践这个思路的一个做法是: 将 std::swap 针对 Widget 特化。下面是基本构想, 但目前这个形式无法通过编译:

```

namespace std {
    template<>                                //这是 std::swap 针对
    void swap<Widget>( Widget& a,             // “T 是 Widget” 的特化版本。
                     Widget& b )             //目前还不能通过编译。
    {
        swap(a.pImpl, b.pImpl);              //置换 Widgets 时只要置换它们的
    }                                          // pImpl 指针就好。
}

```

这个函数一开始的 "template<>" 表示它是 std::swap 的一个全特化 (*total template specialization*) 版本, 函数名称之后的 "<Widget>" 表示这一特化版本系针对 "T 是 Widget" 而设计。换句话说当一般性的 swap template 施行于 Widgets 身上便会启用这个版本。通常我们不能够 (不被允许) 改变 std 命名空间内的任何东西, 但可以 (被允许) 为标准 templates (如 swap) 制造特化版本, 使它专属于我们自己的 classes (例如 Widget)。以上作为正是如此。

但是一如稍早我说, 这个函数无法通过编译。因为它企图访问 a 和 b 内的 pImpl 指针, 而那却是 private。我们可以将这个特化版本声明为 friend, 但和以往的规矩不太

一样：我们令 `Widget` 声明一个名为 `swap` 的 `public` 成员函数做真正的置换工作，然后将 `std::swap` 特化，令它调用该成员函数：

```
class Widget {                                //与前同，唯一差别是增加 swap 函数
public:
    ...
    void swap(Widget& other)
    {
        using std::swap;                      //这个声明之所以必要，稍后解释。
        swap(pImpl, other.pImpl);            //若要置换 Widgets 就置换其 pImpl 指针。
    }
    ...
};

namespace std {
    template<>                                //修订后的 std::swap 特化版本
    void swap<Widget>( Widget& a,
                       Widget& b )
    {
        a.swap(b);                            //若要置换 Widgets，调用其
    }                                           //swap 成员函数。
}
```

这种做法不不仅能够通过编译，还与 STL 容器有一致性，因为所有 STL 容器也都提供有 `public` `swap` 成员函数和 `std::swap` 特化版本（用以调用前者）。

然而假设 `Widget` 和 `WidgetImpl` 都是 `class templates` 而非 `classes`，也许我们可以试试将 `WidgetImpl` 内的数据类型加以参数化：

```
template<typename T>
class WidgetImpl { ... };

template<typename T>
class Widget { ... };
```

在 `Widget` 内（以及 `WidgetImpl` 内，如果需要的话）放个 `swap` 成员函数就像以往一样简单，但我们却在特化 `std::swap` 时遇上乱流。我们想写成这样：

```
namespace std {
    template<typename T>
    void swap< Widget<T> >(Widget<T>& a,        //错误！不合法！
                           Widget<T>& b)
    { a.swap(b); }
}
```

看起来合情合理, 却不合法。是这样的, 我们企图偏特化 (partially specialize) 一个 function template (`std::swap`), 但 C++ 只允许对 class templates 偏特化, 在 function templates 身上偏特化是行不通的。这段代码不该通过编译 (虽然有些编译器错误地接受了它)。

当你打算偏特化一个 function template 时, 惯常做法是简单地为其添加一个重载版本, 像这样:

```
namespace std {
    template<typename T>                //std::swap 的一个重载版本
    void swap (Widget<T>& a,            // (注意 "swap" 之后没有 "<...>")
               Widget<T>& b)            //稍后我会告诉你, 这也不合法。
    { a.swap(b); }
}
```

一般而言, 重载 function templates 没有问题, 但 `std` 是个特殊的命名空间, 其管理规则也比较特殊。客户可以全特化 `std` 内的 templates, 但不可以添加新的 templates (或 classes 或 functions 或其他任何东西) 到 `std` 里头。`std` 的内容完全由 C++ 标准委员会决定, 标准委员会禁止我们膨胀那些已经声明好的东西。啊呀, 所谓“禁止”可能会使你沮丧, 其实跨越红线的程序几乎仍可编译和执行, 但它们的行为没有明确定义。如果你希望你的软件有可预期的行为, 请不要添加任何新东西到 `std` 里头。

那该如何是好? 毕竟我们总是需要一个办法让其他人调用 `swap` 时能够取得我们提供的较高效的 template 特定版本。答案很简单, 我们还是声明一个 non-member `swap` 让它调用 member `swap`, 但不再将那个 non-member `swap` 声明为 `std::swap` 的特化版本或重载版本。为求简化起见, 假设 `Widget` 的所有相关机能都被置于命名空间 `WidgetStuff` 内, 整个结果看起来便像这样:

```
namespace WidgetStuff {
    ...                                //模板化的 WidgetImpl 等等。
    template<typename T>                //同前, 内含 swap 成员函数。
    class Widget { ... };

    ...
    template<typename T>                //non-member swap 函数;
    void swap (Widget<T>& a,            //这里并不属于 std 命名空间。
               Widget<T>& b)
    {
        a.swap(b);
    }
}
```

现在，任何地点的任何代码如果打算置换两个 Widget 对象，因而调用 `swap`，C++ 的名称查找法则（name lookup rules；更具体地说是所谓 *argument-dependent lookup* 或 *Koenig lookup* 法则）会找到 `WidgetStuff` 内的 `Widget` 专属版本。那正是我们所需要的。

这个做法对 `classes` 和 `class templates` 都行得通，所以似乎我们应该在任何时候都使用它。不幸的是有一个理由使你应该为 `classes` 特化 `std::swap`（很快我会描述它），所以如果你想让你的“`class` 专属版”`swap` 在尽可能多的语境下被调用，你需得同时在该 `class` 所在命名空间内写一个 `non-member` 版本以及一个 `std::swap` 特化版本。

顺带一提，如果没有像上面那样额外使用某个命名空间，上述每件事情仍然适用（也就是说你还是需要一个 `non-member swap` 用来调用 `member swap`）。但，何必在 `global` 命名空间内塞满各式各样的 `class`, `template`, `function`, `enum`, `enumerant` 以及 `typedef` 名称呢？难道你对所谓“得体与适度”失去判断力了吗？

目前为止我所写的每一样东西都和 `swap` 编写者有关。换位思考，从客户观点看看事情也有必要。假设你正在写一个 `function template`，其内需要置换两个对象值：

```
template<typename T>
void doSomething(T& obj1, T& obj2)
{
    ...
    swap(obj1, obj2);
    ...
}
```

应该调用哪个 `swap`？是 `std` 既有的那个一般化版本？还是某个可能存在的特化版本？抑或是一个可能存在的 `T` 专属版本而且可能栖身于某个命名空间（但当然不可以是 `std`）内？你希望的应该是调用 `T` 专属版本，并在该版本不存在的情况下调用 `std` 内的一般化版本。下面是你希望发生的事：

```
template<typename T>
void doSomething(T& obj1, T& obj2)
{
    using std::swap;    //令 std::swap 在此函数内可用
    ...
    swap(obj1, obj2);   //为 T 型对象调用最佳 swap 版本
    ...
}
```

一旦编译器看到对 swap 的调用, 它们便查找适当的 swap 并调用之。C++ 的名称查找法则 (name lookup rules) 确保将找到 global 作用域或 T 所在之命名空间内的任何 T 专属的 swap。如果 T 是 Widget 并位于命名空间 WidgetStuff 内, 编译器会使用“实参取决之查找规则” (argument-dependent lookup) 找出 WidgetStuff 内的 swap。如果没有 T 专属之 swap 存在, 编译器就使用 std 内的 swap, 这得感谢 using 声明式让 std::swap 在函数内曝光。然而即便如此编译器还是比较喜欢 std::swap 的 T 专属特化版, 而非一般化的那个 template, 所以如果你已针对 T 将 std::swap 特化, 特化版会被编译器挑中。

因此, 令适当的 swap 被调用是很容易的。需要小心的是, 别为这一调用添加额外修饰符, 因为那会影响 C++ 挑选适当函数。假设你以这种方式调用 swap:

```
std::swap(obj1, obj2);    //这是错误的 swap 调用方式
```

这便强迫编译器只认 std 内的 swap (包括其任何 template 特化), 因而不再可能调用一个定义于它处的较适当 T 专属版本。啊呀, 某些迷途程序员的确以此方式修饰 swap 调用式, 而那正是“你的 classes 对 std::swap 进行全特化”的重要原因: 它使得类型专属之 swap 实现版本也可被这些“迷途代码”所用 (这样的代码出现在某些标准程序库实现版中, 如果你有兴趣不妨帮助这些代码尽可能高效运作)。

此刻, 我们已经讨论过 default swaps、member swaps、non-member swaps、std::swap 特化版本、以及对 swap 的调用, 现在让我把整个形势做个总结。

首先, 如果 swap 的缺省实现码对你的 class 或 class template 提供可接受的效率, 你不需要额外做任何事。任何尝试置换 (swap) 那种对象的人都会取得缺省版本, 而那将有良好的运作。

其次, 如果 swap 缺省实现版的效率不足 (那几乎总是意味你的 class 或 template 使用了某种 pimpl 手法), 试着做以下事情:

1. 提供一个 public swap 成员函数, 让它高效地置换你的类型的两个对象值。稍后我将解释, 这个函数绝不该抛出异常。
2. 在你的 class 或 template 所在的命名空间内提供一个 non-member swap, 并令它调用上述 swap 成员函数。

3. 如果你正编写一个 `class`（而非 `class template`），为你的 `class` 特化 `std::swap`。并令它调用你的 `swap` 成员函数。

最后，如果你调用 `swap`，请确定包含一个 `using` 声明式，以便让 `std::swap` 在你的函数内曝光可见，然后不加任何 `namespace` 修饰符，赤裸裸地调用 `swap`。

唯一还未明确的是我的劝告：成员版 `swap` 绝不可抛出异常。那是因为 `swap` 的一个最好的应用是帮助 `classes`（和 `class templates`）提供强烈的异常安全性（`exception-safety`）保障。条款 29 对此主题提供了所有细节，但此技术基于一个假设：成员版的 `swap` 绝不抛出异常。这一约束只施行于成员版！不可施行于非成员版，因为 `swap` 缺省版本是以 `copy` 构造函数和 `copy assignment` 操作符为基础，而一般情况下两者都允许抛出异常。因此当你写下一个自定版本的 `swap`，往往提供的不只是高效置换对象值的办法，而且不抛出异常。一般而言这两个 `swap` 特性是连在一起的，因为高效率的 `swaps` 几乎总是基于对内置类型的操作（例如 `pimpl` 手法的底层指针），而内置类型上的操作绝不会抛出异常。

请记住

- 当 `std::swap` 对你的类型效率不高时，提供一个 `swap` 成员函数，并确定这个函数不抛出异常。
- 如果你提供一个 `member swap`，也该提供一个 `non-member swap` 用来调用前者。对于 `classes`（而非 `templates`），也请特化 `std::swap`。
- 调用 `swap` 时应针对 `std::swap` 使用 `using` 声明式，然后调用 `swap` 并且不带任何“命名空间资格修饰”。
- 为“用户定义类型”进行 `std templates` 全特化是好的，但千万不要尝试在 `std` 内加入某些对 `std` 而言全新的东西。

5

实现

Implementations

大多数情况下,适当提出你的 `classes` (和 `class templates`) 定义以及 `functions` (和 `function templates`) 声明,是花费最多心力的两件事。一旦正确完成它们,相应的实现大多直截了当。尽管如此,还是有些东西需要小心。太快定义变量可能造成效率上的拖延;过度使用转型 (`casts`) 可能导致代码变慢又难维护,又招来微妙难解的错误;返回对象“内部数据之号码牌 (`handles`)”可能会破坏封装并留给客户虚吊号码牌 (`dangling handles`);未考虑异常带来的冲击则可能导致资源泄漏和数据败坏;过度热心地 `inlining` 可能引起代码膨胀;过度耦合 (`coupling`) 则可能导致让人不满意的冗长建置时间 (`build times`)。

所有这些问题都可避免。本章逐一解释各种做法。

条款 26: 尽可能延后变量定义式的出现时间

Postpone variable definitions as long as possible.

只要你定义了一个变量而其类型带有一个构造函数或析构函数,那么当程序的控制流 (`control flow`) 到达这个变量定义式时,你便得承受构造成本;当这个变量离开其作用域时,你便得承受析构成本。即使这个变量最终并未被使用,仍需耗费这些成本,所以你应该尽可能避免这种情形。

或许你会认为,你不可能定义一个不使用的变量,但话不要说得太早!考虑下面这个函数,它计算通行密码的加密版本而后返回,前提是密码够长。如果密码太短,函数会丢出一个异常,类型为 `logic_error` (定义于 C++ 标准程序库,见条款 54):